Step 3 — The Pipeline (final) — Solution

> Back to the step · Hint 1 · Hint 2

Story: `run_match` ties everything together into one function — build the resume store, extract the job description's required skills, match each one against it — plus `main.py` and the integration test against the real model. **If not:** the three separately-verified pieces would never actually meet, and there'd be no single call a caller could make to use them together.

Sample resume and job (`samples/`)

```
# samples/resume.txt
Senior Backend Engineer with 5 years of experience.
Built REST APIs in Python and Django. Deployed services with Docker on AWS.
Led an Agile team of four; strong mentoring and leadership.
```

```
# samples/job.txt
We are hiring a Backend Engineer.
Required: Python, Docker, AWS, Kubernetes, and team leadership.
```

The pipeline

```
# src/pipeline.py
# Build the resume store, extract the job's required skills, then match each
# skill against the resume by distance. The matcher's distance() becomes a
# lambda over the resume collection.
from src.extractor import extract_required_skills
from src.matcher import match_skills
from src.rag.resume_store import build_resume_store
from src.rag.retriever import best_distance
from src.utils.logger import get_logger

def run_match(model, resume_text, job_text, cutoff=0.8):
    logger = get_logger(__name__)
    collection = build_resume_store(resume_text)
    required = extract_required_skills(model, job_text).skills
```

```
    def distance(skill):
        return best_distance(collection, skill)

    report = match_skills(required, distance, cutoff=cutoff)
    logger.info(
        f"matched {len(report.matched)}/{len(required)} skills "
        f"(score {report.score})"
    )
    return report
```

main.py

```
# main.py
import sys

from src.config import load_config
from src.models.llm import get_model
from src.pipeline import run_match


def main():
    if len(sys.argv) < 3:
        print("usage: python main.py <resume-file> <job-file>")
        return
    with open(sys.argv[1]) as resume_file:
        resume_text = resume_file.read()
    with open(sys.argv[2]) as job_file:
        job_text = job_file.read()

    config = load_config()
    model = get_model(config)
    report = run_match(model, resume_text, job_text, config.match_cutoff)
    total = len(report.matched) + len(report.missing)

    print(f"\nJobMatch score: {report.score}%  "
          f"({len(report.matched)} of {total} required skills)")
    print("\nMatched:")
    for skill in report.matched:
        print(f"  - {skill}")
    print("\nMissing:")
    for skill in report.missing:
        print(f"  - {skill}")
```

```
if __name__ == "__main__":
    main()
```

The integration test (real model + embeddings)

```
# tests/integration/test_live_match.py
# Real model + embeddings. Run from the repo root:
#   python -m tests.integration.test_live_match
from src.config import load_config
from src.models.llm import get_model
from src.pipeline import run_match

RESUME = (
    "Built REST APIs in Python and Django. Deployed with Docker on AWS. "
    "Led an Agile team; strong leadership."
)
JOB = "Required: Python, Docker, AWS, Kubernetes, and team leadership."


def test_scores_high_and_finds_the_gap():
    config = load_config()
    model = get_model(config)
    report = run_match(model, RESUME, JOB, config.match_cutoff)
    ok = report.score >= 60 and "Kubernetes" in report.missing
    result1 = "WRONG"
    if ok:
        result1 = "OK"
    print(f"case 1 (high score, Kubernetes gap) -> {result1}")


if __name__ == "__main__":
    test_scores_high_and_finds_the_gap()
```

Expected run:

```
$ python main.py samples/resume.txt samples/job.txt

JobMatch score: 80%  (4 of 5 required skills)

Matched:
  - Python
  - Docker
  - AWS
  - Leadership
```

```
Missing:
  - Kubernetes
```

That's the whole project: the job's skills extracted as a list, the resume made searchable by meaning, and each skill matched by a distance the deterministic engine turns into a score and a gap list. Because `match_skills` only needs a `distance` number, the RAG layer and the decision engine were built and tested separately — and the same shape matches candidates to roles, tickets to experts, or any "find the closest evidence for each requirement" job.